Information Sciences 319 (2015) 132–151

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Prediction-based energy policy for mobile virtual desktop infrastructure in a cloud environment

Tien-Dung Nguyen, Pham Phuoc Hung, Tran Hoang Dai, Nguyen Huu Quoc, Cong-Thinh Huynh, Eui-Nam Huh *

Department of Computer Engineering, Kyung Hee University, Republic of Korea

ARTICLE INFO

Article history:
Received 15 June 2014
Received in revised form 9 February 2015
Accepted 14 February 2015
Available online 20 February 2015

Keywords:
Energy policy
VM state management
mVDI
Remote desktop service

ABSTRACT

Using cloud services from mobile devices has become a growing trend because of its mobility and convenience. However, mobile devices join and leave cloud services more frequently than traditional computers, which causes energy inefficiency in a cloud data center. Waste, in the form of energy and cooling requirements, particularly occurs when a mobile device disconnects from a service, but the cloud servers, known as virtual machines (VMs), continue running. The VMs should transition to lower-power states instead remaining active. However, transition to a lower-power state causes a service delay when users reconnect to the service because VMs in a lower-power state are not ready to serve. Therefore, an efficient energy policy must not only maximize energy savings but also minimize service delays. In this paper, we propose two approaches to energy efficiency: an Instant Energy Policy (IEP) that can quickly find an appropriate low-power state based on a predicted disconnection time and a Prediction-based Energy Policy (PrEP) that determines when to transition VMs to a low-power state and when to return them to the active state based on each users activity history. IEP predicts the unknown disconnection time using the multisize sliding windows workload estimation technique, which supports a non-stationary environment. This method can quickly obtain an energy policy, but it is limited when disconnection time fluctuates widely. PrEP presents an improved approach to achieve an optimal global result with respect to both energy consumption and service delay. Through simulations with a real-world dataset collected by the MIT Human Dynamics Lab, we show that PrEP provides approximately 20% power saving over the benchmark policies while guaranteeing minimal service delay.

© 2015 Elsevier Inc. All rights reserved.

1. Introduction

The mobile computing revolution is changing many aspects of modern life in positive ways. By using mobile devices such as smartphones or tablets, people can easily play and work anytime, anywhere. However, mobile devices have their own limitations in CPU, memory, storage, battery, etc. These limitations cause cloud service performance degradation (including on-demand movies, 3D games, graphical design tools, etc.). Many promising approaches have been proposed to overcome

* Corresponding author.
E-mail addresses: ntiendung@khu.ac.kr (T.-D. Nguyen), hungpham@khu.ac.kr (P.P. Hung), dai.tran@khu.ac.kr (T.H. Dai), quoc@khu.ac.kr (N.H. Quoc), thinhhc@gmail.com (C.-T. Huynh), johnhuh@khu.ac.kr (E.-N. Huh).

http://dx.doi.org/10.1016/j.ins.2015.02.022
0020-0255/© 2015 Elsevier Inc. All rights reserved.

these issues. The Mobile Virtual Desktop Infrastructure (mVDI) [8] is the evolution of virtualization technology for mobile devices. mVDI has the same look and feel as a traditional mobile device, but all its actions happen remotely in a cloud server. On the server side, audio/video output is not delivered to locally attached devices (such as the monitor and speakers) but redirected over the network to the client for display. Therefore, mVDI not only enhances the performance of mobile services but also reduces the CPU and energy demand of mobile devices.

In an mVDI scenario, each customer remotely controls one or more virtual machines (VMs) in a data center. Every VM consumes a specific amount of energy depending on the services running inside and the type of infrastructure (such as Openstack, Xen, and HyperV). If the number of users increases, the data center sees the effect in rising energy consumption and energy costs as the number of running VMs increases. [18] reports that the energy consumption of data centers rose by 56% from 2005 to 2010, and in 2010 accounted for between 1.1% and 1.5% of global electricity use. Apart from high operation costs, this results in substantial carbon dioxide emissions, about 2% of global emissions [13]. In this paper, we focus on reducing data center energy consumption by implementing efficient energy policies [5].

Regardless of the type of services or infrastructure, VMs consume different amounts of energy depending on their operational state (Fig. 1). In the following table, we present the power consumption of each VM state as measured in an Openstack system.

The active state consumes more energy than the paused or suspended state, which are also called lower-power states. We can reduce energy consumption by efficiently switching a VM between the active and lower-power states instead of keeping it in the active state. Herein, we will continue to discover when and how a VM is best transitioned to lower-power states.

Practically, as customers use VDI-based cloud services, they can face unexpected events, such as receiving phone/Kakao/Viber calls or messages, being interrupted by friends or a low battery, experiencing a weak wifi/3G/4G signal or a sudden OS system reboot, etc. Whenever such a thing happens, customers temporarily disconnect their devices from their VMs for a length of time that depends on the interrupting event. On the data center side, the VMs continue running without any connection to mobile devices, which wastes energy. Thus, disconnected VMs should be transitioned to lower-power states to reduce energy consumption.

However, the transition of VM states causes service delay. For example, imagine a person using cloud services who receives an incoming phone call. The user temporarily disconnects the services and receives the incoming call. The network monitor module detects that the VM has become disconnected and can be switched to a lower-power state. A service delay occurs when the user finishes the call and tries to reconnect the service because the VMs are in a lower-power state and not ready to serve. The delay time is indicated by the time needed to transition from a lower-power state to an active state. Fig. 1 shows that the delay for the paused and suspended states is 2 s and 6 s, respectively. Intuitively, VMs consuming less power have to spend more time to return to the active state. Maximizing energy savings and minimizing service delays are the challenges that will be addressed in this paper.

The most common power management policy is a timeout policy [23] which manages VM states based on time. VMs automatically transition to lower-power states after one given time and return to an active state after another given time. This solution is not optimal because the service time and the disconnection time are not fixed values. A better idea is known

VM State	Power (W)	Time to return active state (s)
Active	55.7	-
Paused	44.5	2
Suspended	43.6	6

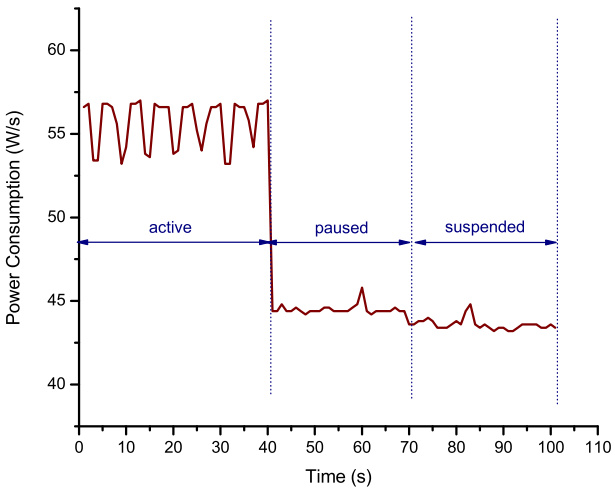

Fig. 1. Power consumption of server with one VM.

as predictive policy [25,17] which is applied in electronic systems. The device immediately transitions to a lower-power state if the predictor estimates that the idle period will be long enough. We here apply a prediction-based policy to solve the energy problem in a cloud environment: the Instant Energy Policy (IEP) provides an instant decision to transition VMs to lower-power states by applying a multisize sliding windows prediction technique [19] to estimate the disconnection time at the moment a user disconnects the service. IEP can quickly obtain a policy, but it is limited by inaccuracy. Because it completely depends on its prediction technique, IEP is not efficient when disconnection times fluctuate widely.

Therefore, we propose a second approach inspired by the work of Simunic et al. [24] to improve IEP using a renewal process. In [24], Simunic et al. proposed a power policy based on stochastic models that use distributions to describe the inter-arrival time, the service time, and the time it takes a device to transition among its states. Using these stochastic models, they obtain the time to transition to lower-power states. However, applying this approach in a cloud environment does not guarantee an optimal result because of the delay when users reconnect to the services. Users have to wait on the transition from lower-power states to the active state, which is called service delay. Therefore, in this paper, we develop an efficient energy policy from the original one in [24] to apply in a cloud environment: the Prediction-based Energy Policy (PrEP). We carefully establish the stochastic models based on statistical analysis of historical data. Our novel approach, PrEP, defines two transition points: when a VM is to be transitioned to a lower-power state and when it returns to the active state for new requests.

The contributions of this work can be summarized into two proposed energy policies as follows:

1. Instant Energy Policy (IEP) in which we heuristically adapt the algorithm to handle unknown non-stationary disconnection time using a multisize sliding windows workload estimation technique and quickly determine a lower-power state and duration in that state.
2. Prediction-based energy policy (PrEP) that relies on a renewal process to find two transition points, when a VM is transitioned to a lower-power state and when it returns to the active state for new requests, with respect to the expected service delay and energy savings.

We evaluate our energy policies with simulations using the Social Evolution dataset collected by the MIT Human Dynamics Lab [20]. The dataset measures all activities on mobile devices (including incoming and outgoing phone call information) from approximately one hundred users during one year. Each phone call duration (seconds) is considered the duration of the temporary disconnect from the on-going services. Our experiments show that our energy policy outperforms the benchmark algorithms with respect to both service delay and energy savings.

The rest of this paper is organized as follows. Related works are reviewed in Section 2. Section 3 describes the system model. We present our IEP approach in Section 4 and our PrEP approach in Section 5. We show some performance results in Section 6 and conclude in Section 7.

2. Related works

In [2], Beloglazov et al. presented a survey of energy management techniques that includes two main methods: (i) Static power management, that includes all optimization methods applied during design at the circuit, logic, architectural, and system levels [9]; and (ii) Dynamic power management (DPM), which includes methods and strategies for run-time adaptation of a systems behavior according to current resource requirements or any other dynamic characteristics of the systems state.

In this paper, we essentially focus on dynamic component deactivation (DCD) which is a DPM technique applied at the hardware and firmware level. DCD techniques are developed based on the clock gating of parts of an electronic component or complete disabling during periods of inactivity. With this technique, servers transition to a lower-power state when the workload falls below a given threshold. However, such a transition not only leads to delays, which can degrade the performance of system, but also consumes more energy. Therefore, the transition must done only if the inactive period is long enough to compensate for the overhead.

DCD has been proposed in Adaptive Link Rate methods [4]. Gupta and Singh [14] change the state of the network interface and other network devices to sleep mode. They proposed a policy to switch the link state of a VM into a sleep mode if the inter-arrival time is above a defined threshold and to advertise each VMs operational state to its neighbors. Neighbors use a wakeup packet to ensure an active state before transmission of actual data. The same group of authors in [15] proposed a DELS policy to place a link in sleep mode according to its buffer occupancy and the mean of the arrival times of recently received packets. The sleep timer was set based on the estimation of the number of packets likely to arrive in the buffer while the link was in the sleep state. Herrería-Alonso et al. [16] proposed an ALR policy with four states: (a) active, (b) idle, (c) sleep, and (d) transition. With more states, this method can achieve around 75% energy savings.

Other DCD techniques have been presented for the power management of electric devices [7,12]. Elnozahy et al. in [11] presented policies that use various combinations of dynamic voltage scaling and node vary-on/vary-off to reduce the aggregate power consumption of a server cluster during periods of reduced workload. A policy that brings nodes online and takes them offline depending on workload intensity produces significant energy savings, up to 42%.

Some other approaches improve energy efficiency by integrating predictions according to historical data [3,1]. In [26], Srivastava et al. presented a predictive shutdown approach in which they tried to predict the length of idle time based on the computation history and then shut down the processor if the predicted length of idle time justified the cost. In [17],

Hwang and Wu presented a new predictive system-shutdown method to exploit sleep mode operations for energy savings. They use an exponential-average approach to predict an upcoming idle period. They then introduce two mechanisms, prediction-miss correction and prewake-up, to improve the hit ratio and reduce the delay overhead. In [6], Chung et al. introduced a DPM technique to reduce power consumption by selectively shutting down idle components. This technique can adapt to changes in user behavior by using idle period clustering and an adaptive learning tree. Simunic et al. [24] enhance the energy policy of electronic devices by modeling all user request arrivals with exponential distributions and developing two new approaches: (i) using renewal theory, the decision to transition to a low-power state can be made in only one state, and (ii) using a time-indexed semi-Markov decision process, this approach allows for transitions into a lower-power state from any state, but it is more complex than other policies. Douglis et al. [10] applied a share algorithm, a machine learning technique, to determine when to spin a disk down.

However, all the previous approaches need to be revised to apply to a cloud environment that depends on various user behaviors and VM state management. We first applied the multisize sliding window method mentioned in [19] to predict user behavior and make decisions based on those predicted values. We then enhanced the energy policy with PrEP, which applies a fitting distribution to provide predicted-distribution parameters to the renewal model. In the next section, we will develop our system model based on actual measurement results using a realistic dataset.

3. System model

3.1. System architecture

Fig. 2 presents the system architecture including 3 main modules: Network Monitor, Prediction module, and Energy Policy. (1) The Network Monitor module monitors transferred packets from every VM to its corresponding mobile device. We suppose that remote packets involve the port number of a remote protocol, such as a VNC protocol using port 5900 or the Windows Remote Desktop protocol using port 3389. If no remote packet is traversed from a running VM during a given time (Δt), that VM will be considered temporarily disconnected, denoted by the binary value 0. Contrarily, if the Network Monitor can detect a remote packet during Δt, it finds that the VM is being remotely controlled by a customer, denoted by the binary value 1. Fig. 3 presents the remote access states of a VM for a given Δt by binary number $\{0, 1\}$. A consecutive 1s sequence forms an active service state (AtS) (e.g., service from starting time t_0 to t_1). Contrarily, a consecutive

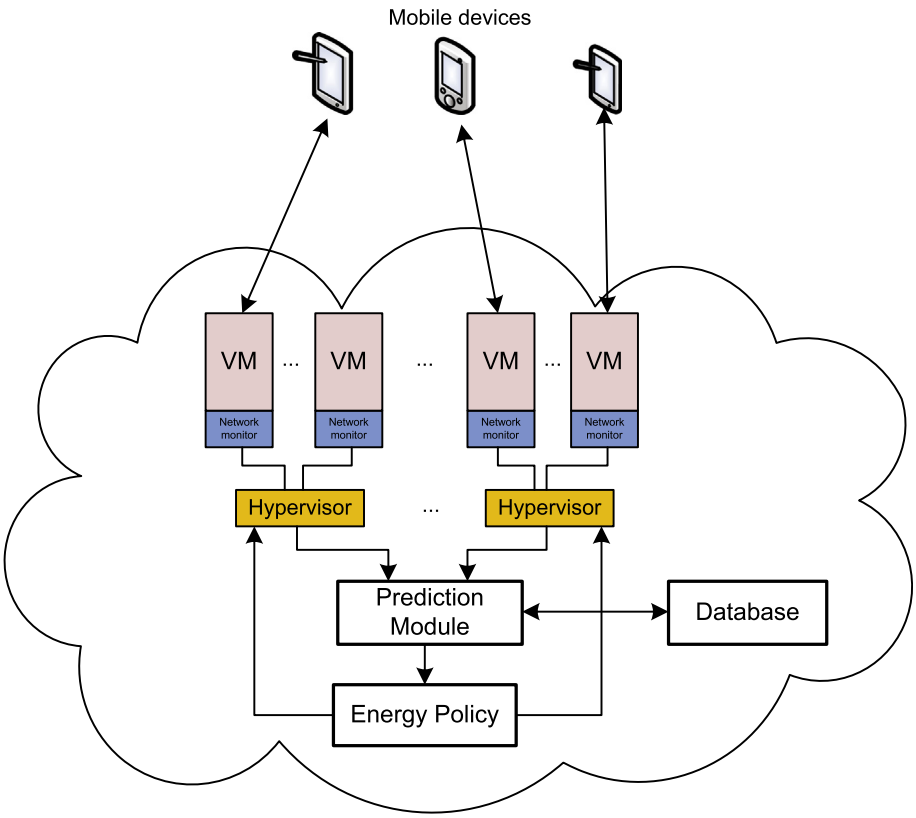

Fig. 2. System architecture.

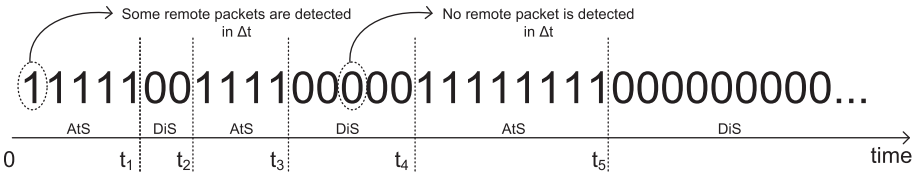

Fig. 3. Remote packet monitoring.

0s sequence forms a disconnection state (DiS) (e.g., no service from time t_3 to t_4). The Network Monitor contains a buffer to store the previous service state of all VMs in the data center. Whenever the Network Monitor detects a mobiles disconnection, it sends a message to the Prediction Module through the hypervisor. (2) The Prediction Module receives a message from the hypervisor, analyzes the historical data, and sends predicted values to the Energy Policy. (3) The Energy Policy decides the best policy for each VM according to the predicted information and sends its decisions to the hypervisor to control each VMs physical state.

3.2. The system model

The physical servers in a data center can be virtualized into multiple VMs remotely controlled by end-users. Depending on the type of hypervisor, each VM probably has different states that consume different amounts of energy. For instance, OpenStack supports four VM states: active, paused, suspended, and shutdown; the active state consumes 55 W, and the suspended state consumes only 43 W. Because the power consumption of the suspended and shutdown states is the same, we consider only three states in our paper: active, paused, and suspended.

We assume that cloud services are real-time, interactive services executed in VMs. A VM can pause the service if the user disconnects and can resume the service (return to active state) as soon as the user reconnects. A service delay occurs when a user reconnects when the VM is not ready to serve.

Let $\bar{t}_{ij}$ and c_{ij} denote expected transition time and power consumption, respectively, when a VM transitions from state i to state j. The detailed notations are presented in the following table.

Notation	Description
$\bar{t}_{ap}$	expected transition time from active state to paused state
$\bar{t}_{pa}$	expected transition time from paused state to active state
$\bar{t}_{as}$	expected transition time from active state to suspended state
$\bar{t}_{sa}$	expected transition time from suspended state to active state
$\bar{t}_{ps}$	expected transition time from paused state to suspended state
$\bar{t}_{A}$	expected service time in one cycle
c_{ap}	power consumption of transition from active state to paused state
c_{pa}	power consumption of transition from paused state to active state
c_{as}	power consumption of transition from active state to suspended state
c_{sa}	power consumption of transition time from suspended state to active state
c_{ps}	power consumption of transition time from paused state to suspended state
c_{a}	power consumption in active state
c_{p}	power consumption in paused state
c_{s}	power consumption in suspended state

4. Instant Energy Policy (IEP)

IEP makes a decision as soon as a user disconnects the service. IEP first predicts the disconnection time and then determines only one lower-power state based on the predicted disconnection time and given system information.

4.1. Disconnection time prediction

As described above, our system has a Network Monitor to monitor user activities. It sends a request to the Predictor Module as soon as a user disconnects from the service. Our Predictor estimates how long the user will remain disconnected from the service (disconnection time) based on that users historical data. In this paper, we apply the multisize sliding window (MsSW) workload estimation technique proposed by Luiz et al. [19] in a cloud environment. This technique is proposed here to predict workload in a non-stationary environment.

MsSW includes three main parts: previous state buffer, window selector, and windows. In our Predictor, disconnection time is converted to ranges. Fig. 4 considers three ranges ($R = 3$) as follows: $0 < r_1 \leqslant 10$ (seconds), $10 < r_2 \leqslant 60$ (seconds), and $r_3 > 60$ (seconds). Each range r_i corresponds with one window W_i. Window is defined as an array, and each piece of content in a window is considered an element of that array. Let W_i^k denote the content of position k in window W_i, and let the maximum size of the window be $l_{w_{max}}$. The previous state buffer stores the range of disconnection state $r(t-1)$ at $(t-1)$ which allows the window selector to choose the window. According to the contents of the selected window, our Predictor will predict the disconnection time at t. For example, if a user disconnected service for 20 s (r_2) at the previous disconnection event, the window selector will choose window W_2 to calculate the transition probability between the ranges of disconnection time. After disconnection, the system will update the actual disconnection time in the selected window by shifting the content of the selected window one position to the right to store the actual disconnection time (converted to range value) in the first position: $W_i^{k+1} = W_i^k, \forall k = 1, 2, \ldots, l_{w_{max}}$; the system will then discard the rightmost content $W_i^{l_{w_{max}}}$ and stores $s_r(t)$ in W_i^1. Let $\hat{p}_{ij}(t,m)$ denote the probability of a transition from state i (at time $t-1$) to state j at time t, and calculate it as follows:

$$\hat{p}_{ij}(t,m) = \frac{\sum_{k=1}^{l(m)} (W_i^k(t) == j)}{l(m)}, \tag{1}$$

where "==" is the equivalence operation, i.e., the result is "1" if the arguments are equal, and "0" otherwise; $l(m)$ is the size of the sliding window elements indicated by the estimated window size index m, with $1 \leqslant m \leqslant l_{w_{max}}$. The precision of MsSW is directly proportional to the size of $l(m)$. However, the amount of historical data is huge, and we cannot use it all. We thus use parameter $l(m)$ to limit the window size and accelerate the calculation. To evaluate the precision of the probability estimates, the next step is to estimate the variance $S(i,j,t,m)$ of the probability estimates obtained from every updated estimate window:

$$S(i,j,t,m) = \frac{1}{l(m)-1} \sum_{k=1}^{l(m)} \left(E_{ijm}^k(t) - \hat{p}_{ij}(t, l(m)) \right)^2, \tag{2}$$

where $E_{ijm}^k(t)$ is the selected window at index k with range r_i at event $(t-1)$ and range r_j at event t, and $\hat{p}_{ij}(t, l(m))$ is the mean value of the probability estimates calculated from the selected window W_i of length $l(m)$.

To determine which values of $l(m)$ can be considered low enough, a function of acceptable variance is defined [19]:

$$V_{ac}\left(\hat{p}_{ij}(t, l(m))\right) = \frac{\hat{p}_{ij}(t, l(m))\left(1 - \hat{p}_{ij}(t, l(m))\right)}{l(m)}. \tag{3}$$

The output of this method is the estimation transition probability $\hat{p}_{ij}$ from range r_j at event $(t-1)$ to range r_i at event t. Assuming that disconnection time currently belongs to range r_i, the estimated disconnection time is calculated as:

$$\hat{d} = \sum_{j=1}^{R} \hat{r}_j \hat{p}_{ij}$$

where $\hat{r}_j$ is calculated using the average of actual values stored in the historical data corresponding with range r_j.

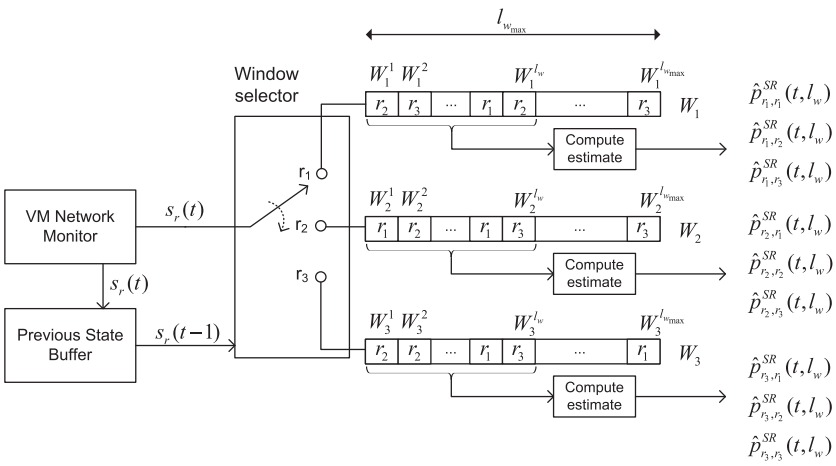

Fig. 4. Example of multisize sliding window with three states.

4.2. Instant algorithm

After obtaining the disconnection time $\hat{d}$, IEP determines the state to which the VM is to be transitioned by the following algorithm:

Algorithm 1. Instant Energy Policy

Require: $\hat{d}, \bar{t}_{ap}, \bar{t}_{pa}, \bar{t}_{as}, \bar{t}_{sa}$
  if $\hat{d} \geqslant \bar{t}_{as} + \bar{t}_{sa}$ **then**
    state $\leftarrow$ suspended state
    duration $\leftarrow \hat{d} - (\bar{t}_{as} + \bar{t}_{sa})$
  else if $\hat{d} \geqslant \bar{t}_{ap} + \bar{t}_{pa}$ **then**
    state $\leftarrow$ paused state
    duration $\leftarrow \hat{d} - (\bar{t}_{ap} + \bar{t}_{pa})$
  else
    state $\leftarrow$ active state
  end if
  return state, duration

As shown in Algorithm 1, making a decision using IEP is quite simple. Only one state with a corresponding duration time can be determined as soon as a user disconnects. Therefore, IEP can provide a decision quickly in real time. However, the Predictor determines the accuracy of IEP. In particular, the MsSW technique cannot provide a highly precise prediction if the distribution of disconnection time fluctuates widely. We thus continue to present an improved energy policy, PrEP, in the next section.

5. Prediction-Based Energy Policy (PrEP)

In PrEP, the policy is also decided based on historical data. However, unlike in IEP, PrEP uses renewal theory with respect to the disconnection time distribution.

5.1. Disconnection time distribution prediction

PrEP is constructed by using a renewal process based on a stochastic model. We thus predict the distribution of disconnection time instead of the exact value used by IEP. Disconnection time distribution can be evaluated using probability distribution fitting techniques. In this paper, we apply fitting distribution techniques built into the statistics toolbox of Matlab [21] to predict the distribution in the Social Evolution dataset. The disconnection time is obtained from phone call durations in call logs and defined as historical data for our simulations.

Fig. 5 illustrates the CDF and PDF of an exponential fit distribution and normal fit distribution. Immediately, we can see that the exponential distribution better fits the dataset than the normal distribution. Thus, using the exponential distribution

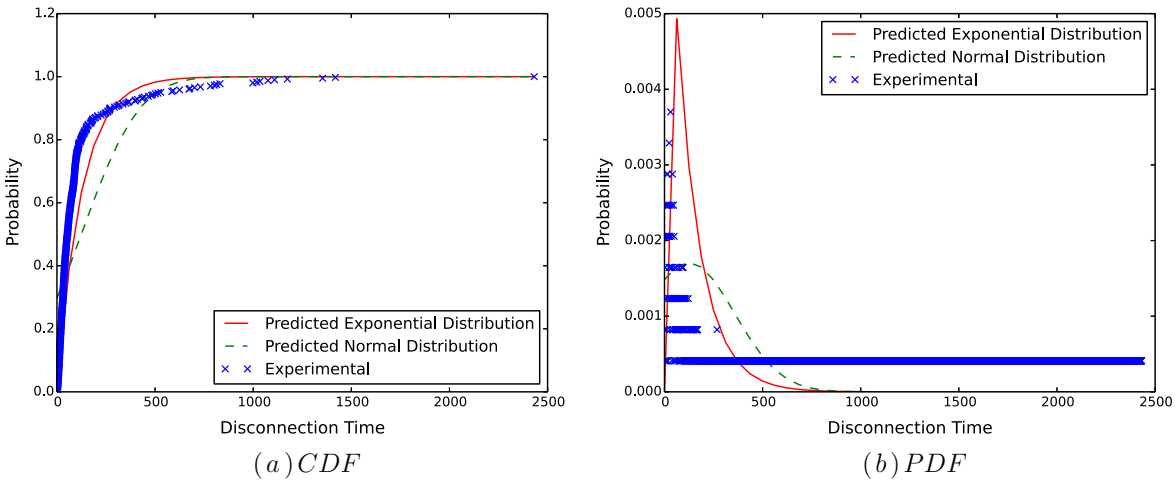

Fig. 5. Disconnection time distribution caused by a phone call.

to predict will provide more accurate results. In our algorithm, we choose a suitable fit distribution depending on the expected error, which can be calculated as the average difference between the predicted value and the actual value. The fit probability distribution with the lowest expected error obtains the most accurate results.

5.2. Energy policy based on renewal theory

Renewal theory studies stochastic systems whose evolution over time contains a set of renewals or regeneration times. Formally, renewal is a counting process such that the time until the first event occurs has some distribution F, the time between the first and second event has, independently of the time of the first event, the same distribution F, and so on. When an event occurs, a renewal is said to have taken place. In this case, the distributions are not history dependent, i.e., they are memory-less.

An energy policy starts to run when a user disconnects from a VM. The VM is supposed to transition to the paused state after a given time. Therefore, the cycle is supposed to start with the paused state. Let one renewal process be defined as the complete cycle of transition from the paused state through the other states and back to the paused state. One cycle can include any number of states: paused, suspended, and active. The cycle can happen in three main cases:

1. Paused → Suspended → Paused → Active → Paused: this is the full cycle of the renewal process. The user reconnects after the VM transitions to the suspended state and returns to the paused state.
2. Paused → Suspended → Active → Paused: the user reconnects after the VM transitions to the suspended state and before it returns to the paused state.
3. Paused → Active → Paused: the VM transitions to the active state without switching to the suspended state because the user quickly reconnects to the service.

In PrEP, we state that the VM transitions to the active state as soon as the user reconnects and automatically transitions to the paused state when the user disconnects. During disconnection time, the policy will determine when the VM is to be transitioned to the suspended state and when it returns to the paused state. Thus, we determine two transition points, from the paused to the suspended state and from the suspended to the paused state. As we measure the system, the time to transition from the paused state to the active state is much less than from the suspended state to the active state. If the policy only considers the first point (when a VM is to be transitioned from paused to suspended), the VM is always suspended when the user reconnects, which increases delay when resuming service. Therefore, we define the second time point, when a VM is to be transitioned from the suspended state to the paused state to wait for the user to reconnect. Intuitively, defining the second point should reduce the delay but require more power consumption. Therefore, we consider the tradeoff between power savings and delay time in our energy management policy.

We assume random variable $\mathcal{X}$ to specify the transition from the paused state to the suspended state and a second time point for the transition from the suspended state to the paused state. $\mathcal{X}$ defines the first point as a value in $[0, h, 2h, \ldots, jh, \ldots, Nh]$, where j is an index, h is the smallest unit time, and N is the maximum time before the system goes to the suspended state. The second point is taken as a value in $[jh, (j+1)h, \ldots, (j+\delta)h, Mh]$, where δ is an index and M. M is the maximum time before the system returns to the paused state.

Let $p(j,\delta)$ be defined as the probability of issuing a command to go to the suspended state at jh and return to paused at $(j+\delta)h$, $d(j,\delta)$ as the time penalty (or service delay) a user incurs in the transition to the suspended state, $e(j,\delta)$ as the expected energy consumption, and $t(j,\delta)$ as the expected time of a cycle. We formulate an optimization problem to minimize the expected delay under a power constraint ($E_{constraint}$). The power constraint equals the maximum power needed to minimize the delay time. The optimization problem including the unknown $p(j,\delta)$ is presented as follows:

$$\begin{aligned} \min \quad & \frac{\sum_j \sum_\delta p(j,\delta) d(j,\delta)}{\sum_j \sum_\delta p(j,\delta) t(j,\delta)} \\ s.t. \quad & \sum_j \sum_\delta p(j,\delta) e(j,\delta) = E_{constraint} \\ & \sum_j \sum_\delta p(j,\delta) = 1 \\ & p(j,\delta) \geqslant 0 \quad \forall j,\delta \end{aligned} \tag{4}$$

In the following sections, we present how to obtain the expected time of a cycle $t(j,\delta)$, the expected energy consumption $e(j,\delta)$, and the expected service delay $d(j,\delta)$.

5.2.1. Expected cycle time

Let β be defined as the time at which a user reconnects to the service. The distribution of β can be calculated by the distribution of disconnection time, denoted by general probability distribution P. We also define $I(a)$ as the indicator function, $I(a) = 1$ if a is true, and 0 otherwise.

$$\begin{aligned} t(j,\delta) = {} & E[t(j,\delta)I(\beta \leqslant jh)|X = (jh,(j+\delta)h)] + E[t(j,\delta)I(jh < \beta \leqslant (j+\delta)h)|X = (jh,(j+\delta)h)] \\ & + E[t(j,\delta)I((j+\delta)h < \beta \leqslant (j+\delta)h + \bar{t}_{sa})|X = (jh,(j+\delta)h)] \\ & + E\big[t(j,\delta)I\big((j+\delta)h + \bar{t}_{sa} < \beta \leqslant (j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}\big)|X = (jh,(j+\delta)h)\big] \\ & + E\big[t(j,\delta)I\big(\beta > (j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}\big)|X = (jh,(j+\delta)h)\big] \end{aligned} \tag{5}$$

Each term in Eq. (5) is described more detail in the five cases below. In our scenario, we do not consider queue in each state; in other words, when a user reconnects to the service when the VM is not in the active state, no job or task can be sent to the VM at that time, and the user has to wait until the VM reaches the active state (delay time).

Case 1: $Y = \beta \leqslant jh$.

The user reconnects to the service when the VM is paused, before it transitions to the suspended state.

$$E[t(j,\delta)I(\beta \leqslant jh)|X = (jh,(j+\delta)h)] = \sum_{s=0}^{j} shP(\beta = sh) + P(Y)\big(\bar{t}_{pa} + E(t_A) + \bar{t}_{ap}\big) \tag{6}$$

where $E(t_A)$ is the time the user is expected to be connected to the VM before the next disconnection.

Case 2: $Y = jh < \beta \leqslant (j+\delta)h$.

The user reconnects to the service when the VM is in the suspended state.

$$E[t(j,\delta)I(jh < \beta \leqslant (j+\delta)h)|X = (jh,(j+\delta)h)] = \sum_{s=j}^{j+\delta} shP(\beta = sh) + P(Y)\big(jh + \bar{t}_{ps} + \bar{t}_{sa} + E(t_A) + \bar{t}_{ap}\big) \tag{7}$$

Case 3: $Y = (j+\delta)h < \beta \leqslant (j+\delta)h + \bar{t}_{sa}$.

Because there is no direct transition from the suspended state to the paused state, the VM must transition to the active state and then to the paused. In this case, the user reconnects to the service while the VM is transitioning from the suspended state to the active state. For this case, after the transition to the active state, the VM will stay active to provide service instead of completing the transition to the paused state.

$$E[t(j,\delta)I((j+\delta)h < \beta \leqslant (j+\delta)h + \bar{t}_{sa})|X = (jh,(j+\delta)h)] = P(Y)\big((j+\delta)h + \bar{t}_{sa} + E(t_A) + \bar{t}_{ap}\big) \tag{8}$$

Case 4: $Y = (j+\delta)h + \bar{t}_{sa} < \beta \leqslant (j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}$.

In this case, the user reconnects to the service while the VM is transitioning from the active state to the paused state.

$$E\big[t(j,\delta)I\big((j+\delta)h + \bar{t}_{sa} < \beta \leqslant (j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}\big)|X = (jh,(j+\delta)h)\big] = P(Y)\big((j+\delta)h + \bar{t}_{sa} + \bar{t}_{pa} + E(t_A) + 2\bar{t}_{ap}\big) \tag{9}$$

Case 5: $Y = \beta > (j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}$.

The user reconnects to the service while the VM is in the paused state.

$$E[t(j,\delta)I(Y)|X = (jh,(j+\delta)h)] = P(Y)\begin{bmatrix} (j+\delta)h + \bar{t}_{sa} + \bar{t}_{pa} + E(t_A) + 2\bar{t}_{ap} \\ +E\big(\beta - \big((j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}\big)\big)I(Y) \end{bmatrix} \tag{10}$$

5.2.2. Expected energy consumption

First, we divide the renewal cycle into 9 smaller intervals of VM states and transitions (Fig. 6); each interval consumes a different amount of energy. Obviously, we can obtain the expected energy consumption if we calculate the expected time intervals. In the following section, we present how to formulate the expected time of each interval. In our scenario, if a user reconnects to the VM while it is transitioning, the user must wait until the transition finishes.

- Paused: The expected time spent in the paused state can be determined by intervals 1 and 6. Interval 1 occurs when a user reconnects to the VM before jh, ($\beta \leqslant jh$) and is calculated by $\sum_{s=0}^{j} shP(\beta = sh)$. If the user reconnects to the VM after jh, ($\beta > jh$), the time the VM is in the paused state is obviously jh. Meanwhile, interval 6 occurs when the VM is in the paused state when the user reconnects, calculated by $P(\beta > \vartheta)E(\beta - \vartheta)I(\beta > \vartheta)$, where $\vartheta = (j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap}$.
- Paused → Suspended: This transition occurs during interval 2 only if the user reconnects after jh. Even if the user reconnects while the VM is transitioning, the system must wait until the transition is completed to return the VM to the active state. The expected transition from the paused to the suspended state is calculated as $\bar{t}_{ps}P(\beta > jh)$.

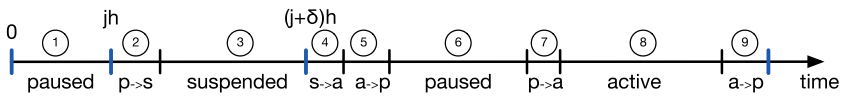

Fig. 6. A completed renewal cycle with 3 states: active-paused-suspended.

- Suspended: The suspended time is measured in interval 3 and can occur in 2 cases. The first case is when a user reconnects to the VM after jh and before $(j+\delta)h$ and is calculated by $\sum_{s=j}^{j+\delta} shP(\beta = sh)$. The second is when the user reconnects to the VM after $(j+\delta)h$ and is calculated by $(\delta h - \bar{t}_{ps})^{+} P(\beta > (j+\delta)h)$.
- Suspended → Active: This transition always occurs when a user reconnects after jh (interval 4): $\bar{t}_{sa} P(\beta > jh)$.
- Active: This period always exists in the renewal cycle because the user remains connected to the VM until the next disconnection (interval 8). This period is measured using the expected value from the historical data ($E(t_A)$).
- Active → Paused: This transition can occur in two cases. The first case always occurs in the last interval of a renewal cycle when the VM transitions to the paused state following the users disconnection (interval 9). The second case occurs when the VM is to be transitioned to the paused state (interval 5) and is calculated by $\bar{t}_{ap} P(\beta > (j+\delta)h)$.
- Paused → Active: This transition occurs only when the VM stays in the paused state to be transitioned to the active state as soon as the user reconnects to the VM (interval 7). Its expected transition time is calculated by $\bar{t}_{pa} P(\beta > (j+\delta)h + \bar{t}_{sa})$.

The expected energy consumption can be calculated as follows:

$$e(j,\delta) = \sum_{i=1}^{7} e_i \tag{11}$$

where e_i is described in the table below:

Notation	Case	Expected Energy Consumption
e_1	Paused	$c_i \begin{bmatrix} \sum_{s=0}^{j} shP(\beta = sh) + \\ jhP(\beta > jh) + \\ P(\beta > \vartheta) E(\beta - \vartheta) I(\beta > \vartheta) \end{bmatrix}$
e_2	Paused → Suspended	$c_{ps}\left[\bar{t}_{ps} P(\beta > jh)\right]$
e_3	Suspended	$c_s \begin{bmatrix} \sum_{s=j}^{j+\delta} shP(\beta = sh) + \\ (\delta h - \bar{t}_{ps})^{+} P(\beta > (j+\delta)h) \end{bmatrix}$
e_4	Suspended → Active	$c_{sa}[\bar{t}_{sa} P(\beta > jh)]$
e_5	Active	$c_a E(t_A)$
e_6	Active → Paused	$c_{ap}\left[\bar{t}_{ap} P(\beta > (j+\delta)h) + \bar{t}_{ap}\right]$
e_7	Paused → Active	$c_{pa}\left[\bar{t}_{pa} P(\beta > (j+\delta)h + \bar{t}_{sa})\right]$

5.2.3. Expected service delay

The penalty occurs when user reconnects the VM but the VM is not ready to use. The delayed time is calculated from the time when user reconnects the VM until VM is to be transitioned to active state. The time VM returns to active state has different values, depending on the current VM state. We categorize into 5 states to calculate the expected delayed time.

- Paused: The user reconnects to the VM when the VM is in the paused state. The delay time is the expected transition time from the paused state to the active state ($\bar{t}_{pa}$). This case possibly occurs in either interval 1 ($P(\beta \leqslant jh)$) or interval 6 ($P(\beta > \vartheta)$).
- Paused → Suspended: The delay in this case occurs because the user reconnects to the VM while it is being transitioned from the paused state to the suspended state (interval 2); the expected delay time can be calculated as $P(jh < \beta \leqslant jh + \bar{t}_{ps})$.
- Suspended: The VM is suspended and takes the delay time to return to the active state when the user reconnects (interval 3); the expected delay time can be calculated as $P(jh + \bar{t}_{ps} < \beta \leqslant (j+\delta)h)$.
- Suspended → Active: As explained above, to transition from the suspended state to the paused state, the VM has to return to the active state and then transition to the paused state. If a user reconnects to the VM while it is being transitioned from the suspended state to the active state (interval 4), the expected delay time can be calculated as $P((j+\delta)h < \beta \leqslant (j+\delta)h + \bar{t}_{sa})$.
- Active → Paused: This transition is shown in intervals 5 and 9. A user who reconnects in interval 5 must wait until the VM transitions to the paused state and then to the active state. The probability of interval 5 is $P((j+\delta)h + \bar{t}_{sa} < \beta \leqslant \vartheta)$. A user who reconnects in interval 9, is shifted to the next renewal cycle, which is equivalent to reconnecting at the beginning of interval 1 (see Paused above).

The expected delay time is calculated as follows:

$$d(j,\delta) = \sum_{i=1}^{5} d_i \tag{12}$$

where d_i is described in the table below:

Notation	Case	Expected delay time
d_1	Paused	$[P(\beta \leqslant jh) + P(\beta > \vartheta)]\bar{t}_{sa}$
d_2	Paused → Suspended	$P(jh < \beta \leqslant jh + \bar{t}_{ps}).$ $\left[I\left(jh < \beta \leqslant jh + \bar{t}_{ps}\right)E\left(jh + \bar{t}_{ps} - \beta\right) + \bar{t}_{sa}\right]$
d_3	Suspended	$P\left(jh + \bar{t}_{ps} < \beta \leqslant (j+\delta)h\right)\bar{t}_{sa}$
d_4	Suspended → Active	$P((j+\delta)h < \beta \leqslant (j+\delta)h + \bar{t}_{sa}).$ $\begin{bmatrix} I((j+\delta)h < \beta \leqslant (j+\delta)h + \bar{t}_{sa}). \\ E((j+\delta)h + \bar{t}_{sa} - \beta) \end{bmatrix}$
d_5	Active → Paused	$P((j+\delta)h + \bar{t}_{sa} < \beta \leqslant \vartheta).$ $\begin{bmatrix} I((j+\delta)h + \bar{t}_{sa} < \beta \leqslant \vartheta). \\ E\left((j+\delta)h + \bar{t}_{sa} + \bar{t}_{ap} - \beta\right) \\ +\bar{t}_{pa} \end{bmatrix}$

5.2.4. Problem formulation

Let $x[j,\delta] = p(j,\delta)\Big/\sum_j \sum_\delta p(j,\delta)t(j,\delta)$, and $y[j,\delta] = 1\Big/\sum_j \sum_\delta p(j,\delta)t(j,\delta)$. The optimization problem shown in (4) can then be transformed into an LP problem as follows:

$$\begin{aligned} \min \quad & \sum_j \sum_\delta d(j,\delta)x[j,\delta] \\ s.t. \quad & \sum_j \sum_\delta e(j,\delta)x[j,\delta] - y[j,\delta]E_{constraint} = 0 \\ & \sum_j \sum_\delta t(j,\delta)x[j,\delta] = 1 \\ & \sum_j \sum_\delta x[j,\delta] - y[j,\delta] = 0 \\ & x[j,\delta] \geqslant 0 \quad \forall j,\delta \\ & y[j,\delta] > 0 \quad \forall j,\delta \end{aligned} \tag{13}$$

To solve the LP shown in Eq. (13), we transform the optimization problem into a two-dimension matrix:

$$\begin{aligned} \min \quad & tr(\mathbf{D}^T\mathbf{X}) \\ s.t. \quad & tr(\mathbf{E}^T\mathbf{X}) - YE_{constraint} = 0 \\ & tr(\mathbf{T}^T\mathbf{X}) = 1 \\ & \mathbf{1}^T\mathbf{X}\mathbf{1} - Y = 0 \\ & \mathbf{X}_{ij} \geqslant 0 \quad \forall i,j \\ & Y > 0 \end{aligned} \tag{14}$$

where $tr(A)$ is a trace of matrix A defined as the sum of the elements on the main diagonal (the diagonal from the upper left to the lower right). We can obtain X and Y by solving the LP presented in Eq. (14) and infer the value of the corresponding $x[j,\delta]$ and $y[j,\delta]$. The probability of transition to the suspended state from the paused state at time jh and return to the paused state from the suspended state at time $(j+\delta)h$, $p(j,\delta)$ can be calculated as:

$$p(j,\delta) = \frac{x(j,\delta)}{y(j,\delta)} \tag{15}$$

5.3. PrEP implementation

In the optimization problem, the parameters mainly depend on the historical data of an individual users activities, which are collected after each cycle of the renewal process. PrEP can be implemented in two ways. (i) the first implementation is to update j and δ after every cycle of renewal process. After one cycle, the system collects the actual disconnection time and adds it to the historical data. PrEP is then re-processed to obtain an updated j and δ from the updated historical data. This implementation has a high overhead because it requires multiple reevaluations, and the results can change insignificantly. (ii) PrEP can also be reevaluated after a given time or a given number of cycles, even though the system continues to update the historical data after every cycle. This implementation reduces the overhead and maximizes the systems accuracy.

Our PrEP energy policy is a table of probability $p(j,\delta)$ obtained by solving the LP problem shown in (14). As we defined above, $p(j,\delta)$ is the probability that VM transitions to the suspended state at time jh and returns to paused state after δh. We

then calculate the table of cumulative probability $P(j,\delta)$ based on the probability distribution described with $p(j,\delta)$. In PrEP, we define a random probability value (*RPV*) as a condition to choose j and δ. The time interval for which the policy gives a cumulative probability $P(j,\delta)$ greater than the *RPV* gives the times at which the VM is to be transitioned to the suspended state (jh) and returned to the paused state (δh).

In the LP problem shown in (14), the time complexity depends on the given range of j and δ. Considering a long interval time makes the time to complete the LP problem extremely long. Therefore, we evaluate δ and obtain the cumulative probability table according to every value of j. We can find the value of δ^* from the cumulative probability table with respect to the given *RPV*. For example, the below table presents the cumulative probability of user 91 with $j = 36$. Assume $RPV = 0.5, \delta^*$ is 110 for the case $j = 36$.

j	δ	Transition probability
36	1	0.0000
	...	0.0000
	106	0.0001
	107	0.0001
	108	0.0004
	109	0.4568
	110	0.9997
	111	0.9999
	112	0.9999
	113	0.9999
	114	1.0000

We then calculate the energy consumption and delay time for each value of j with the best choice of δ^* using the function $Find_energy(j,\delta^*)$ and $Find_delay(j,\delta^*)$, respectively. Eventually, we determine the value of j^* by finding the minimal values for both delay time and energy consumption. Our algorithm is briefly shown as follows.

Algorithm 2. PrEP: Determine j and δ

Require: $length_j, RPV$
  for $j \leftarrow 1 : length_j$ **do**
    Solve LP Problem, obtain cumulative probability table
    Find best choice of δ based on RPV
    $Energy[j] \leftarrow Find_energy\ (j,\delta^*)$
    $Delay[j] \leftarrow Find_delay\ (j,\delta^*)$
    $Delta[j] \leftarrow \delta^*$
  end for
  $j^* \leftarrow$ Find j based on Energy and Delay
  $\delta^* \leftarrow Delta[j^*]$
  return j^*, δ^*

5.4. Power consumption and transition time measurement

In this paper, we measure power consumption and transition time based on multiple VMs. We installed Openstack (Icehouse version) and created 10 VMs on our HP Server (HP Server – 4 Cores Intel(R) Xeon(R) CPU E3–1220 V2 @ 3.10 GHz – 32 GB RAM). Each VM runs an infinite loop bash script to print its hostname to keep the CPU in the busy state. The results of power consumption and transition time are measured by an electrical power metering device (model SJPM-C16).

As shown in Fig. 7, we measured two VM states: suspended (red line) and paused (black line). In Fig. 7(a), we activated VMs while all remaining VMs were suspended. In Fig. 7(b), we activated VMs while all remaining VMs were paused. Fig. 7 shows that power consumption in the paused state is higher than in the suspended state and depends on CPU usage. In both cases, CPU usage increased from zero (server idle) to 72%. This value is configured in OpenStack such that total CPU usage of guests is limited to approximately 72% to guarantee host stability. Therefore, when the system increases the number of running VMs to 4, CPU usage increases steadily because of the infinite loop running on each VM. Until VM5, the total CPU usage is around 70–72%, no matter how many VMs are used because the system schedules jobs from each VM to guarantee that CPU usage will not exceed the maximum allowance value. Corresponding with the range of CPU usage, the power consumption in this circumstance also adjusts from 41 to 73 watts, as described in the following table.

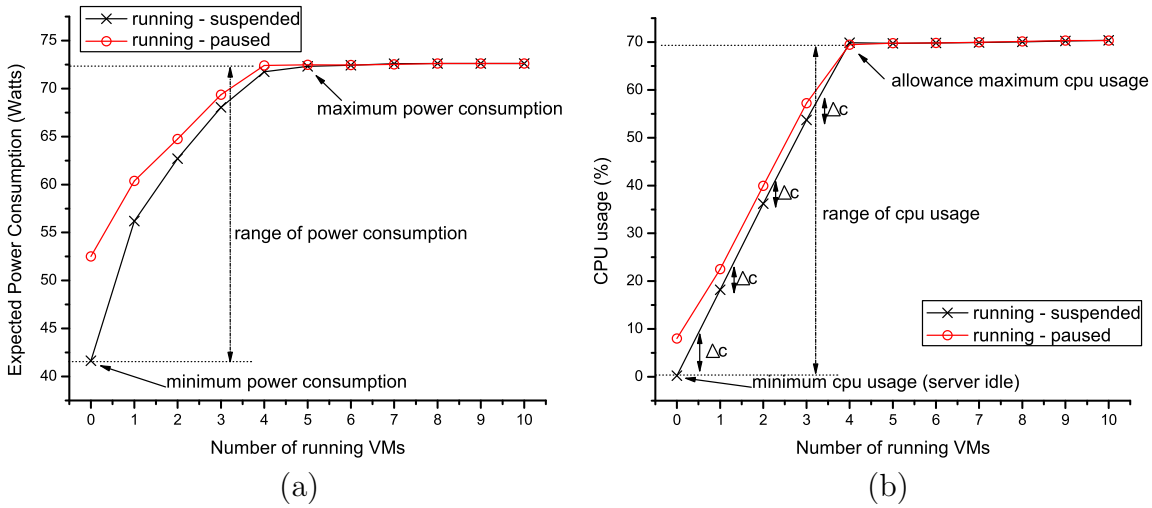

Fig. 7. Power consumption and CPU usage.

	CPU usage		Power consumption	
	min	max	min	max
HP Server - 4 Cores Intel(R) Xeon(R) CPU E3–1220 V2 @ 3.10 GHz 32 GB RAM	0.25%	72%	41 W	73 W

We present the relationship between power consumption and CPU usage in Fig. 8. Obviously, the power consumption is directly proportional to the CPU usage. Let $f(\cdot)$ be the function of power consumption and CPU usage, c be the CPU usage of server at time t. The power consumption p at time t can be calculated as: $p(t) = f(c)$.

How should power consumption be calculated when one VM transitions to the paused or suspended state? Let C_t be the total CPU usage of all VMs at current time t, C_{vm} be the CPU usage of VM that needs to be paused or suspended, and C_r be the total CPU usage of the remaining VMs. We can calculate power consumption as follows:

$$P_{\text{suspended}} = \begin{cases} 0 & \text{if } C_r = C_t \\ f(C_r) & o.w. \end{cases} \quad \text{and} \quad P_{\text{paused}} = \begin{cases} 0 & \text{if } C_r = C_t \\ f(C_r) + f(\Delta C) & o.w. \end{cases}$$

where ΔC is the expected additional CPU usage between pausing and suspending (as shown in Fig. 7).

We continue to study the effects of the number of VMs on transition time. In this experiment, we successively measured transition times by increasing the number of running VMs from 1 to 10. Fig. 9 presents the transition time from the active state to the suspended state and vice versa: from 4.5 s to 7.5 s. The transition time from running state to paused state and

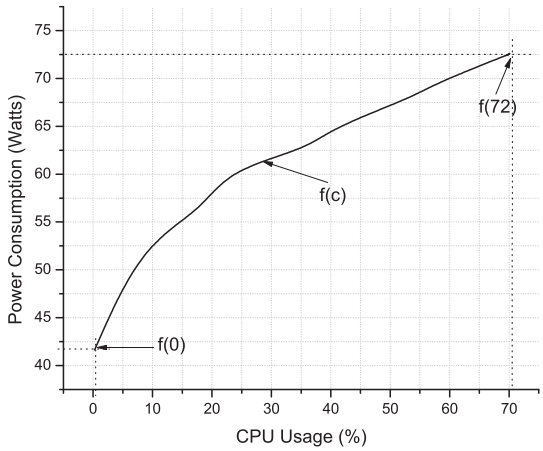

Fig. 8. Effects of CPU usage on power consumption.

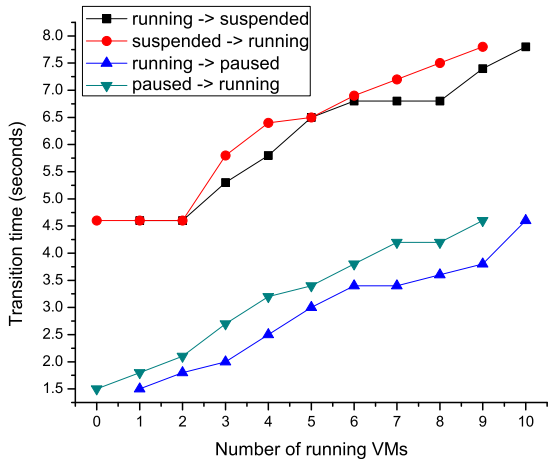

Fig. 9. Transition time.

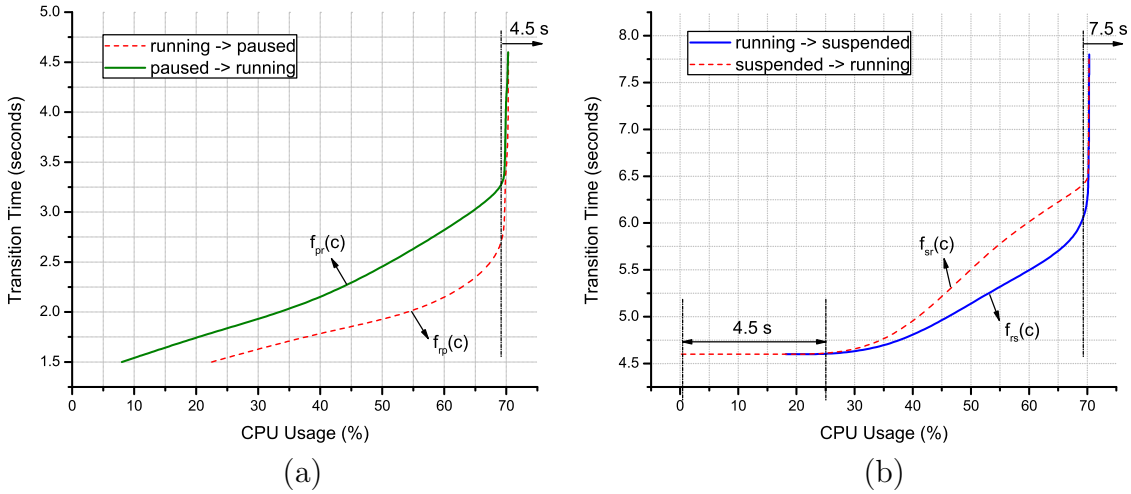

Fig. 10. Effects of CPU usage on transition time.

vice versa adjusts from 1.5 s to 4.5 s. The transition time from the active state to the paused state and vice versa adjusts from 1.5 s to 4.5 s. Obviously, transition from active to paused and vice versa is faster than the transition from active to suspended and vice versa.

Logically, transition time depends on CPU usage, as illustrated in Fig. 10, from which functions for the transition time based on CPU usage can be obtained. The transition time from the active to the paused state (t_{rp}) and vice versa (t_{pr}) is inferred as follows:

$$t_{rp} = \begin{cases} f_{rp}(c) & c < 69 \\ 4.5 & o.w. \end{cases} \quad \text{and} \quad t_{pr} = \begin{cases} f_{pr}(c) & c < 69 \\ 4.5 & o.w. \end{cases}$$

We can infer the transition time from the active to the suspended state (t_{rs}) and vice versa (t_{sr}) as follows:

$$t_{rs} = \begin{cases} 4.5 & c < 25 \\ f_{rs}(c) & 25 \leqslant c < 69 \\ 7.5 & o.w. \end{cases} \quad \text{and} \quad t_{sr} = \begin{cases} 4.5 & c < 25 \\ f_{sr}(c) & 25 \leqslant c < 69 \\ 7.5 & o.w. \end{cases}$$

6. Simulation results

6.1. Simulation environment

In our simulation, we used an HP Server (HP Server – 4 Cores Intel(R) Xeon(R) CPU E3–1220 V2 @ 3.10 GHz 32 GB RAM) and installed a CentOS 6.5 server with OpenStack (Icehouse version) [22] as a virtualization platform. We installed VMs with

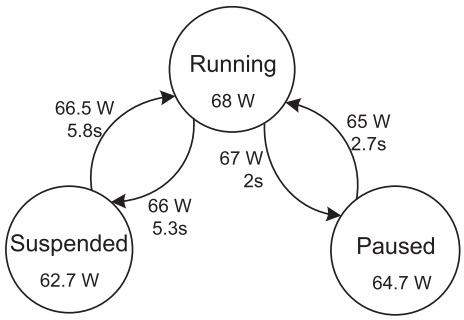

Fig. 11. Power consumption and transition time in OpenStack system.

the lightweight Linux Operating System (Cirros). Each VM runs an infinite loop bash script to print its hostname that keeps the CPU in the busy state. We presented the means we used to measure power consumption and transition time in the previous section. However, for simplicity, we assume that the CPU usage of the server is maintained near 50% in our experiment. Therefore, we can easily calculate the power consumption and transition time for the 3 VM states, as shown in Fig. 11.

As discussed earlier, we used the Social Evolution dataset [20]], which contains information about incoming and outgoing phone calls whose duration we considered as the disconnection time. We implemented our approach using Matlab with the dataset to evaluate performance based on energy consumption and service delay. In the simulations, we choose $P_{constraint}$ as the average power consumption of all VM states.

In this section, we successively evaluate prediction accuracy, energy consumption, and delay time of IEP and PrEP.

6.2. Prediction accuracy

Because the core of IEP is MsSW estimation, to evaluate the accuracy of IEP, we must evaluate the accuracy of the MsSW mechanism. First, we loaded the whole dataset, including the whole duration of voice calls, into memory as the historical data. We then estimated the duration from the first event and compared the estimate with the actual duration of the corresponding disconnection event in the memory.

On the other hand, PrEP uses fit distribution to estimate the cumulative distribution of user behavior based on the renewal model. To evaluate the accuracy of PrEP, we evaluated the results of delay time, assuming that a user has to wait for certain time (the penalty) to reconnect to the service after disconnecting for a given time. In this experiment, we set the penalty as the transition time from the paused state to the active state. If the delay time is less than the penalty, the accuracy is 100%, and otherwise it is 0%. We calculated the accuracy of IEP and PrEP using the average accuracy of all users in the dataset.

Fig. 12(a) presents the accuracy of IEP, 55.5% and Fig. 12(b) presents the accuracy of PrEP, 92.2%. The accuracy of IEP is low because it depends completely on the estimation technique while user behavior is non-stationary. Meanwhile, PrEP achieves

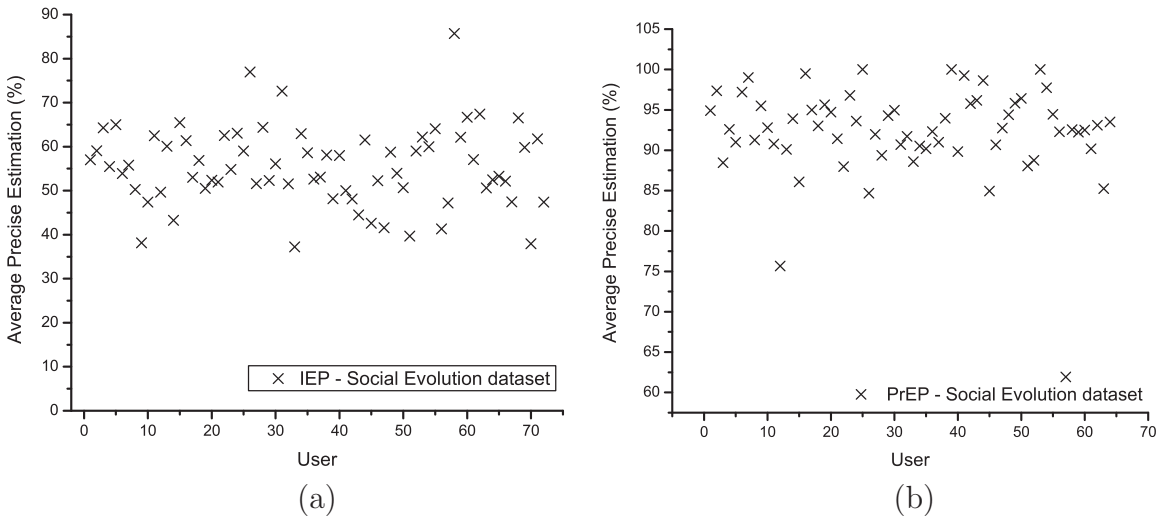

Fig. 12. Accuracy of IEP and PrEP.

better accuracy than IEP because it does not make decisions based primarily on predicted values. PrEP uses the renewal model defined as an optimization problem to find the optimal solution.

6.3. Energy consumption and service delay

First, we compare the energy savings and service delay between IEP and PrEP. The results, shown in Fig. 13, show that the average energy consumption of PrEP is lower than both IEP and Wo-EP (without energy policy). The average service delay of PrEP (approximately 3 s) is also lower than that of IEP (approximately 5 s).

Generally, IEP has lower performance than PrEP. However, IEP can provide a quick decision to reduce energy consumption and guarantee minimal service delay at an acceptable level. Therefore, IEP can be applied to services that do not have strict real-time requirements.

In the next simulations, we studied the performance of PrEP, considering the time intervals j and δ, denoted PrEP(j, δ). We compared PrEP with three other methods: Wo-EP, PrEP(j), and EP-WoS. Wo-EP is implemented without any energy policy; every VM is always in the active state. PrEP(j) is a prediction-based energy policy that considers only interval time j, without considering δ; PrEP(j) thus determines when a VM is to be transitioned to the suspended state, where it stays until the users next reconnection. The last method is EP-WoS, an energy policy without a suspended state; EP-WoS considers only the active and paused states. Figs. 14–16 present the performance comparison between our approach PrEP(j, δ) and the other energy policies on their expected energy consumption (in joules), expected power consumption (in watts), and expected delay times.

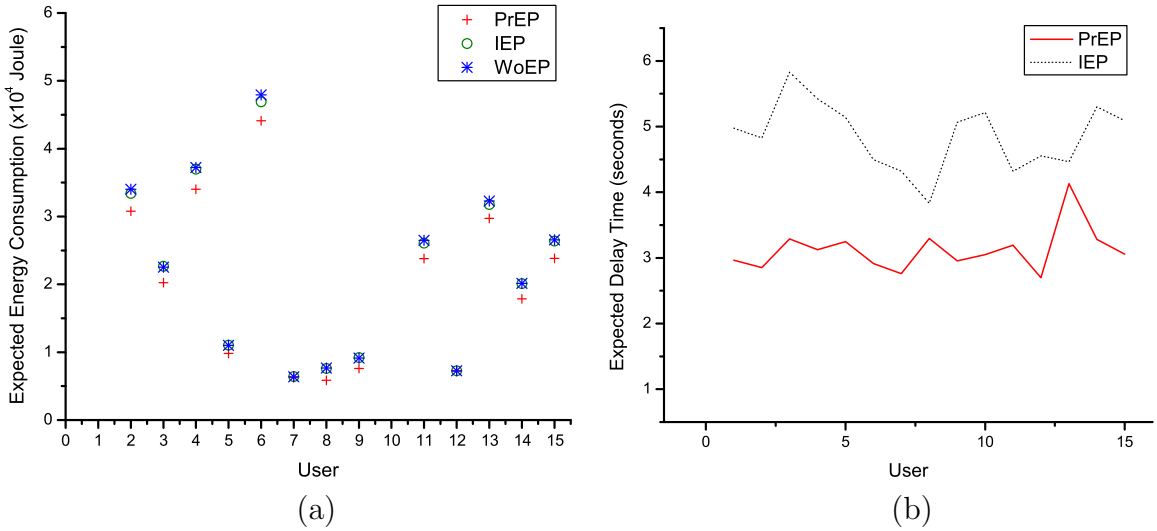

Fig. 13. IEP and PrEP performance.

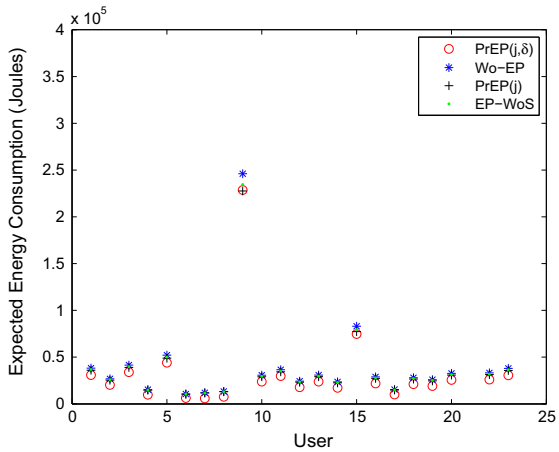

Fig. 14. Comparison of expected energy consumption.

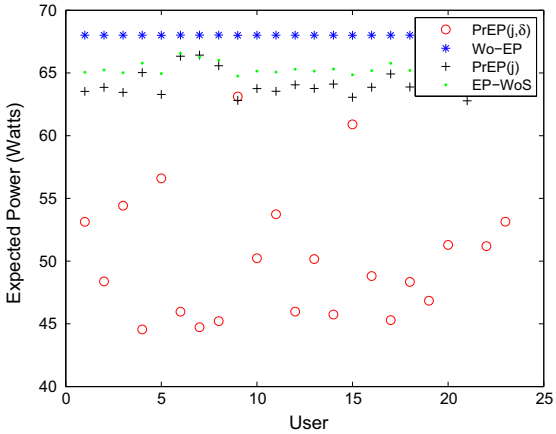

Fig. 15. Comparison of expected power consumption.

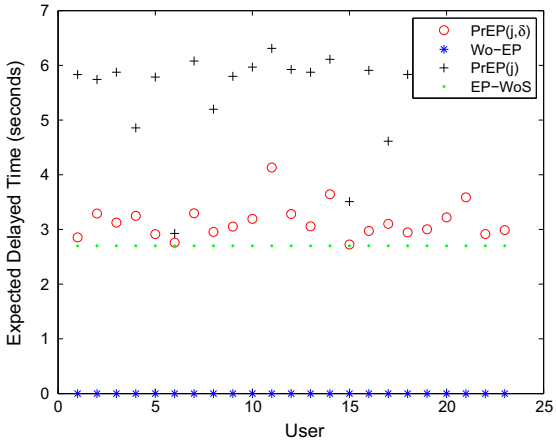

Fig. 16. Comparison of expected delayed time.

The results show that Wo-EP consumes the most energy and power. Power consumption is defined as the energy consumed in one cycle. Obviously, Wo-EP has no delay because it has no transition. The three remaining methods (PrEP(j, δ), PrEP(j) and EP-WoS) consume about same amount of energy, as shown in Fig. 14. However, as shown in Fig. 15, their power consumption actually differs quite a bit. The expected power consumption of EP-WoS is approximately 65.28 W, higher than PrEP (51.44 W). Because EP-WoS uses only the paused state without the suspended state, its delay time is exactly the transition time from the paused state to the active state (2.7 s in our simulation), lower than PrEP (3.13 s). PrEP(j, δ) outperforms PrEP(j) on both expected power consumption and delay time thanks to its transition to the paused state (as shown in Fig. 15). PrEP(j) determines when a VM should transition to the suspended state and then waits for the user to reconnect. Therefore, its delay time is the transition time from the suspended state to the active state. On the other hand, our approach (PrEP(j, δ)) also considers when to transition from the suspended state back to the paused state. Our approach analyzes the historical data to predict the expected disconnection time and thereby determines how long the VM stays in the suspended state. PrEP(j, δ) may increase energy consumption, it however decreases the service delay.

Eventually, we conducted an experiment with all users in the dataset, with the average results shown briefly in the following table.

Method	Total energy consumption ($\times 10^3$ Joule)	Delay (s)	Power consumption (W)
PrEP(j, δ)	54.6	3.13	51.44
Wo-EP	62.9	0	68.01
PrEP(j)	58.7	5.57	63.99
EP-WoS	60.2	2.7	65.28

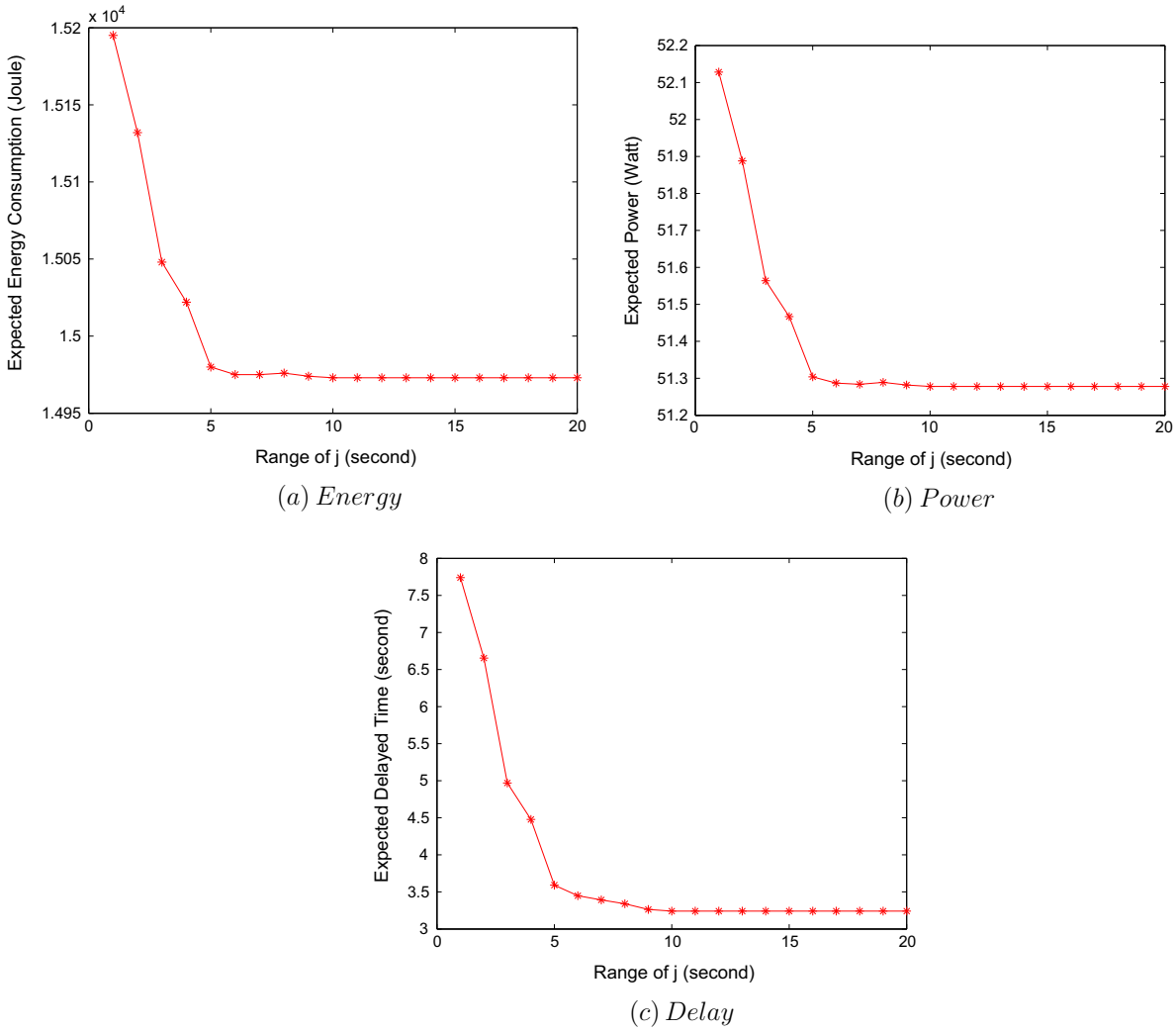

Fig. 17. The impact of range j.

Our approach PrEP(j, δ) consumes 51.44 W/s and guarantees the expected delay around 3.13 s. From the perspective of both power consumption and delay, our approach outperformed the other energy policies.

6.4. Time complexity of PrEP

PrEP decides when a VM is to be transitioned to the suspended state (jh) and when it should return to the paused state (δh). The time complexity of PrEP thus depends on the range of the parameters j and δ. Therefore, choosing those ranges is important not only to reduce the time complexity but also to achieve the best result. To determine the best range for each parameter, we first experimented with one user and observed the effects of parameters j and δ successively.

As shown in Figs. 17 and 18, the expected energy consumption, delay time, and power gradually reach a saturated point as the range of j and δ increase. In the first simulation, we increased the range of j from 1 to 20 s and set the range of δ to the expected value of the disconnection time; the optimal expected value of energy consumption, power, and delay time decreased and saturated at $j \leqslant 5$ s. Thus, the maximum time to keep a VM in the paused state is 5 s. If we attempt to keep VM in the paused state for more than 5 s, the optimal results will be achieved at $j \in [0, 5]$. In the second simulation, we adjusted the range of δ from 1 to 1000 s. The expected energy and power consumption nearly saturated at $\delta = 200$. The expected delay time is changed as the range of δ increased, but the changes were negligible. Normally, the expected delay time will increase with the range of δ, but it decreased in our simulation because, by adjusting j to achieve an optimal expected delayed time, the disconnection time can almost be less than j. However, the effect is insignificant and we can limit the range of δ to reduce the time complexity. Considering our server equipped with CPU 4 Cores, RAM 4 GB, the PrEP time complexity with 60 s as the range of j, and 1000 s as the range of δ is approximately 21 min. This is too much time to calculate j and δ for the next disconnection event. Therefore, based on our analysis, we can reduce the searching range of j and δ while

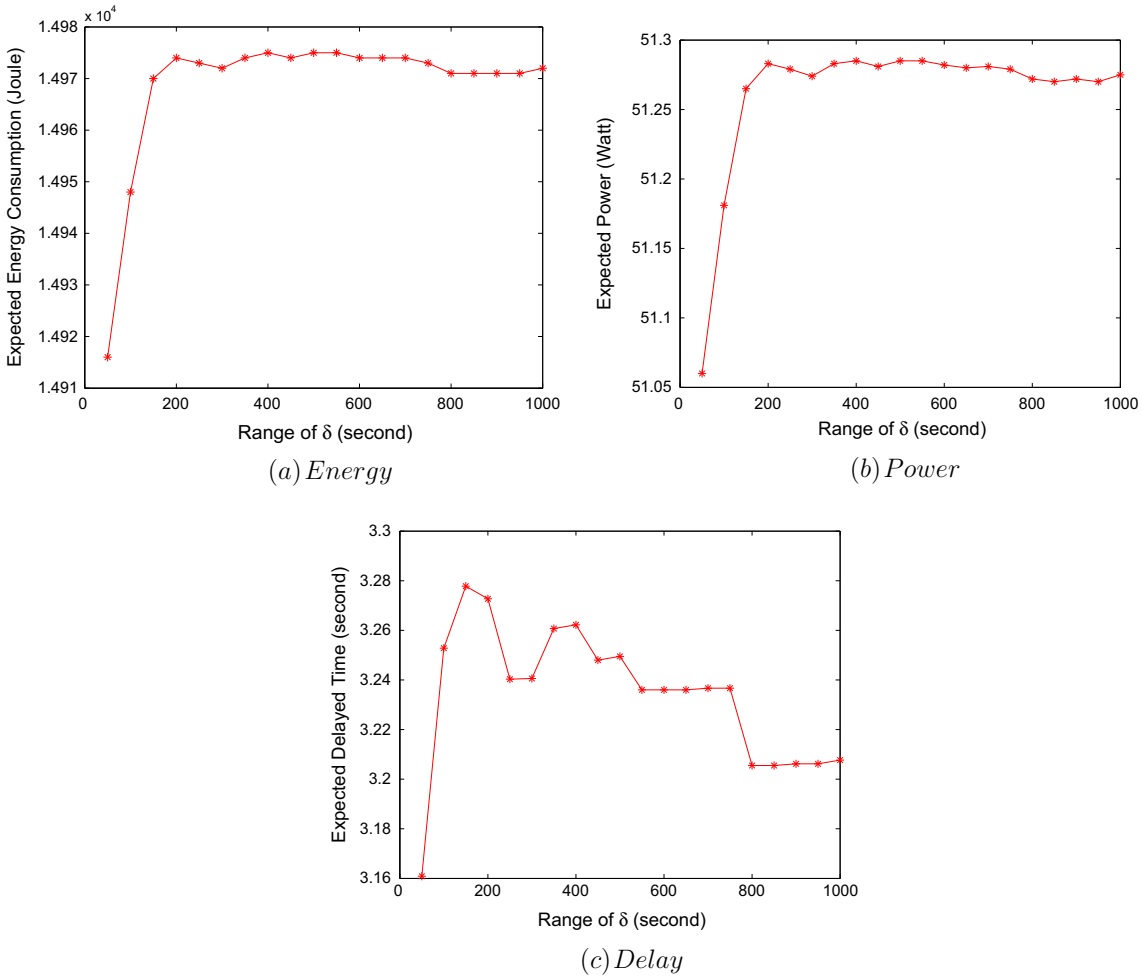

Fig. 18. The impact of range δ.

achieving optimal results. If we reduce the ranges more, time processing will continue to decrease, but the accuracy will also decrease. Therefore, in practice depending on the number of VMs, we can adjust the searching range to meet real-time requirements.

7. Conclusion

In this paper, we propose two approaches to energy efficiency based on the prediction of future user behaviors. Our Instant Energy Policy (IEP) decides quickly but has limited accuracy when disconnection times fluctuate widely. A better approach is our Prediction-based Energy Policy (PrEP), which determines when VMs should transition to lower-power states and when they should to be returned the active state based on the historical data of each users activities. Through simulations with real-world dataset, we show that our PrEP approach outperforms the benchmark policies by providing approximately 20% power saving and guaranteeing minimal service delay. In practice, with a large number of VMs managed using our policy, service providers can save a large amount of energy, which will greatly reduce costs for both end-users and service providers.

Acknowledgement

This research was supported by the MSIP (Ministry of Science, ICT&Future Planning), Korea, under the ITRC (Information Technology Research Center) support program (NIPA-2014(H0301-14-1020)) supervised by the NIPA (National IT Industry Promotion Agency).

References

[1] S. Albers, Energy-efficient algorithms, Commun. ACM 53 (5) (2010) 86–96.
[2] A. Beloglazov, R. Buyya, Y.C. Lee, A. Zomaya, A Taxonomy and Survey of Energy-Efficient Data Centers and Cloud Computing Systems, 2010.
[3] L. Benini, A. Bogliolo, G. De Micheli, A survey of design techniques for system-level dynamic power management, IEEE Trans. Very Large Scale Integration (VLSI) Syst. 8 (3) (2000) 299–316.
[4] K. Bilal, S.U. Khan, S.A. Madani, K. Hayat, M.I. Khan, N. Min-Allah, J. Kolodziej, L. Wang, S. Zeadally, D. Chen, A survey on green communications using adaptive link rate, Cluster Comput. 16 (3) (2013) 575–589.
[5] K. Bilal, S. Malik, S. Khan, A. Zomaya, Trends and challenges in cloud datacenters, IEEE Cloud Comput. 1 (1) (2014) 10–20.
[6] E.-Y. Chung, L. Benini, G. De Micheli, Dynamic power management using adaptive learning tree, in: Proceedings of the 1999 IEEE/ACM International Conference on Computer-aided Design, ICCAD '99, IEEE Press, Piscataway, NJ, USA, 1999, pp. 274–279.
[7] G. Debnath, K. Debnath, R. Fernando, The pentium processor-90/100, microarchitecture and low power circuit design, in: Proceedings of the 8th International Conference on VLSI Design, VLSID '95, IEEE Computer Society, Washington, DC, USA, 1995, p. 185.
[8] L. Deboosere, B. Vankeirsbilck, P. Simoens, F. De Turck, B. Dhoedt, P. Demeester, Cloud-based desktop services for thin clients, IEEE Internet Comput. 16 (6) (2012) 60–67.
[9] S. Devadas, S. Malik, A survey of optimization techniques targeting low power vlsi circuits, in: Proceedings of the 32Nd Annual ACM/IEEE Design Automation Conference, DAC '95, ACM, New York, NY, USA, 1995, pp. 242–247.
[10] F. Douglis, P. Krishnan, B.N. Bershad, Adaptive disk spin-down policies for mobile computers, in: Proceedings of the 2nd Symposium on Mobile and Location-Independent Computing, MLICS '95, USENIX Association, Berkeley, CA, USA, 1995, pp. 121–137.
[11] E.M. Elnozahy, M. Kistler, R. Rajamony, Energy-efficient server clusters, in: Proceedings of the 2nd Workshop on Power-Aware Computing Systems, 2002, pp. 179–196.
[12] S. Gary, P. Ippolito, G. Gerosa, C. Dietz, J. Eno, H. Sanchez, Powerpc 603, a microprocessor for portable computers, IEEE Des. Test 11 (4) (1994) 14–23.
[13] I. Gatner, Gatner estimates ICT industry accounts for 2 percent of global CO_2 emissions, Gatner Press Release, 2007.
[14] M. Gupta, S. Singh, Greening of the internet, in: Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, SIGCOMM '03, ACM, New York, NY, USA, 2003, pp. 19–26.
[15] M. Gupta, S. Singh, Dynamic ethernet link shutdown for energy conservation on ethernet links, in: IEEE International Conference on Communications, 2007, ICC '07, June 2007, pp. 6156–6161.
[16] S. Herrería-Alonso, M. Rodríguez-Pérez, M. Fernández-Veiga, C. López-García, Opportunistic power saving algorithms for ethernet devices, Comput. Netw. 55 (9) (2011) 2051–2064.
[17] C.-H. Hwang, A.-H. Wu, A predictive system shutdown method for energy saving of event-driven computation, in: 1997 IEEE/ACM International Conference on Computer-Aided Design, 1997, Digest of Technical Papers, November 1997, pp. 28–32.
[18] J. Koomey, Growth in Data Center Electricity Use 2005 to 2010, Analytics Press, Oakland, CA, 2011.
[19] S. Luiz, A. Perkusich, A. Lima, Multisize sliding window in workload estimation for dynamic power management, IEEE Trans. Comput. 59 (12) (2010) 1625–1639.
[20] A. Madan, M. Cebrian, S. Moturu, K. Farrahi, A. Pentland, Sensing the health state of a community, IEEE Pervasive Comput. 11 (4) (2012) 36–45.
[21] Matlab, Fit Probability Distribution, 2014. <Http://www.mathworks.com/help/stats/fitdist.html>.
[22] OpenStack, OpenStack Cloud Software, 2014. <Https://www.openstack.org/>.
[23] D. Ramanathan, R. Gupta, System level online power management algorithms, in: Proceedings Design, Automation and Test in Europe Conference and Exhibition 2000, 2000, pp. 606–611.
[24] T. Simunic, L. Benini, P. Glynn, G. De Micheli, Event-driven power management, IEEE Trans. Comput.-Aided Des. Integrated Circ. Syst. 20 (7) (2001) 840–857.
[25] M. Srivastava, A. Chandrakasan, R. Brodersen, Predictive system shutdown and other architectural techniques for energy efficient programmable computation, IEEE Trans. Very Large Scale Integration (VLSI) Syst. 4 (1) (1996) 42–55.
[26] M.B. Srivastava, A.P. Chandrakasan, R.W. Brodersen, Predictive system shutdown and other architectural techniques for energy efficient programmable computation, IEEE Trans. Very Large Scale Integr. Syst. 4 (1) (1996) 42–55.